JAVA BACKEND + AWS

장대산

API와 DB 규칙의 구현부터
배포와 복구 절차까지

diqkraud@naver.com

github.com/daesan12

01 BACKEND + INFRASTRUCTURE

패밀링크

그룹과 피드 API, DB 제약

S3 직접 업로드 API 구현과 서명 조건 수정

AWS 배포 스크립트 구현

프로젝트 보기 p.02 →

02 INFRASTRUCTURE

잘타슈

데이터 처리와 서비스 실행을 위한 환경 구성

승인형 배포, Jenkins 복구, 백업 검증

프로젝트 보기 p.09 →

BACKEND + INFRASTRUCTURE

패밀링크

가족의 일상과 안부를 연결하는 모바일 서비스

서비스	가족 피드와 일정 공유, AI 통화와 리포트
기간	2026.07.13 – 08.10
팀	6명 (AI 2, 프론트 2, 백엔드 2) 백엔드와 인프라를 함께 담당

직접 맡은 작업

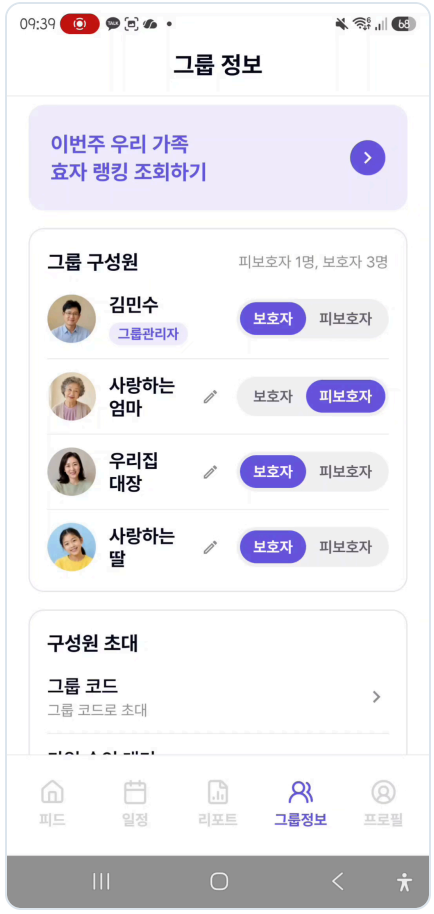
그룹과 피드 API, DB 제약과 일일 적립

S3 업로드 URL 발급과 서명 조건 수정

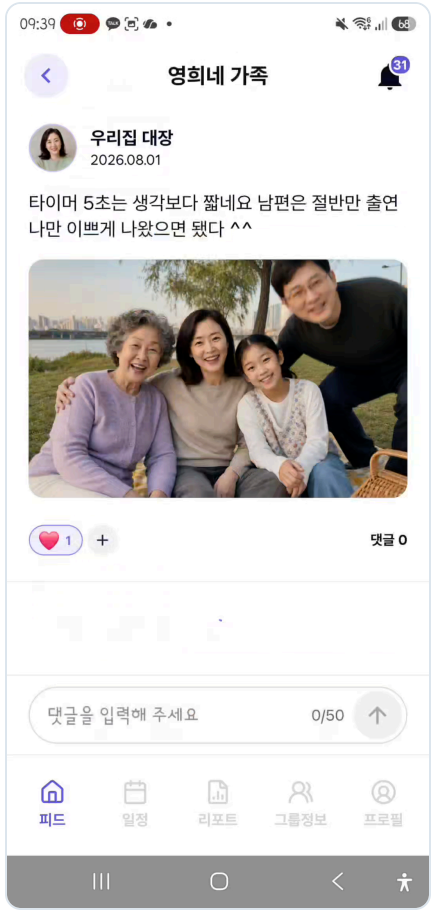
팀 운영 AWS 인프라와 SHA 배포

Java, Spring Boot, JPA, PostgreSQL, Redis

EC2, RDS, S3, ECR, SSM, Jenkins, Docker



그룹 정보와 구성원 역할



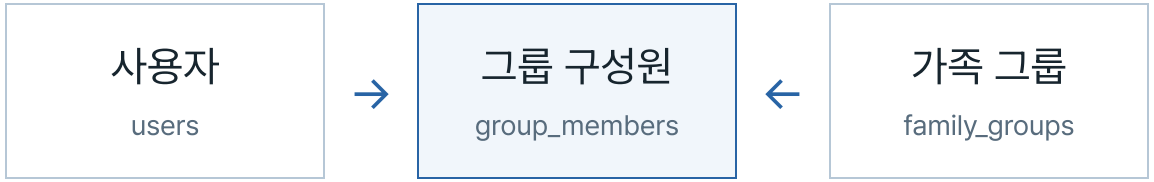
가족 피드와 댓글, 반응

팀 서비스 시연 화면

가족 그룹의 관계를 API 권한과 DB 제약에 반영

그룹마다 역할과 탈퇴 상태를 따로 관리하도록, 합의된 요구사항을 멤버십 스키마와 API로 구현했습니다.

사용자와 그룹 관계



> 게시글 → 댓글과 반응
>
> 그룹에 속한 작성자를 기준으로 연결

API 권한 검사	확인하는 기준
피드 접근과 작성	활성 그룹 구성원
게시글 수정과 삭제	인증 사용자와 작성자 일치
댓글과 반응	게시글 접근과 행위자 권한

활성 관리자 최대 1명 제한

```
CREATE UNIQUE INDEX uq_group_active_manager
  ON group_members (group_id)
  WHERE is_manager = TRUE
    AND left_at IS NULL;
```

API는 요청자의 권한을 검사하고, 이 DB 제약은 그룹당 활성 관리자가 최대 1명만 저장되도록 제한합니다.

API와 스키마 구현

멤버십에 역할, 관리자 여부, 탈퇴 상태를 저장했습니다. 피드와 댓글, 반응 및 개인화 호칭 API를 구현하고, 스키마는 Flyway로 관리했습니다.

페이지 요청을 API 계약대로 검증

QA에서 발견한 페이지 요청 검증과 API 계약의 불일치를 수정했습니다.

보정할 입력과 거부할 입력의 차이

계약은 잘못된 요청을 거부하도록 정했지만, 기존 구현은 음수 page 보정과 과도한 size, 외부 sort를 허용할 수 있었습니다.

입력	반영한 계약
page	0 이상
size	기본 20, 1~100
sort	외부 지정 거부

API마다 같은 입력 기준을 적용하도록 검증을 공통 함수로 모았습니다.
기본값과 허용 범위는 기존 계약을 따릅니다.

잘못된 페이지 요청 거부

```
if (resolvedPage < 0 || resolvedSize < 1 || resolvedSize >
MAX_SIZE || (sort != null && !sort.isEmpty())) {
    throw new BusinessException(ErrorCode.INVALID_REQUEST);
}
return PageRequest.of(resolvedPage, resolvedSize, fixedSort);
```

조회 순서와 범위도 함께 수정

생성 시각이 같을 때는 ID를 보조 정렬 기준으로 사용했습니다.
보관함은 DB 조회 단계에서 필터와 페이지 제한을 적용했습니다.

여러 번 글을 올려도 같은 날 적립은 한 번

그룹 구성원과 KST 날짜를 기준으로 일일 적립 규칙을 구현했습니다.

적립 대상 확인

보호자 역할인지 검사하고
그룹 구성원 단위로 적립합니다.

날짜 기준 통일

`Asia/Seoul` 기준의 날짜를
적립 키에 사용합니다.

DB에서 중복 삽입 처리

구성원과 날짜의 UNIQUE 제약과
`ON CONFLICT`를 함께 사용합니다.

구성원과 날짜 기준 중복 적립 방지

```
insert into carnations (group_id, group_member_id, earned_date)
values (:groupId, :groupMemberId, :earnedDate)
on conflict (group_member_id, earned_date) do nothing
```

게시글과 분리한 적립 기록

게시글을 삭제해도 적립 기록은 유지하도록 두 기록을 분리했습니다.

S3 직접 업로드 구현과 모바일 서명 보완

Presigned URL 발급 API부터 실제 AWS S3 연결, 모바일 전송 조건 수정까지 담당했습니다.

01 직접 업로드 API 구현

기존 API 계약에 따라 Presigned PUT URL 발급 API를 구현했습니다. 앱이 파일을 S3로 직접 전송합니다.

02 실제 S3 저장과 조회 연결

HeadObject로 업로드된 객체의 존재, 크기, MIME을 확인하고 비공개 조회용 Presigned GET URL을 구현했습니다.

영상 길이 검증에는 S3 객체 임시 다운로드가 포함됩니다.

03 모바일 전송과 서명 조건 보완

API 201과 curl PUT 성공에도 앱 업로드가 실패해, 실제 Blob 크기로 URL을 요청하고 같은 Blob을 PUT하도록 수정했습니다. S3 오류도 별도 처리했습니다. 백엔드는 Content-Type을 유지하고 Content-Length를 서명 조건과 반환 헤더에서 제외했습니다.

Content-Length 서명 조건 제거

```
 .contentType(file.contentType())
-.contentLength(file.sizeBytes())
 .build();

+Map.of("Content-Type", file.contentType())
```

서명 헤더 회귀 테스트

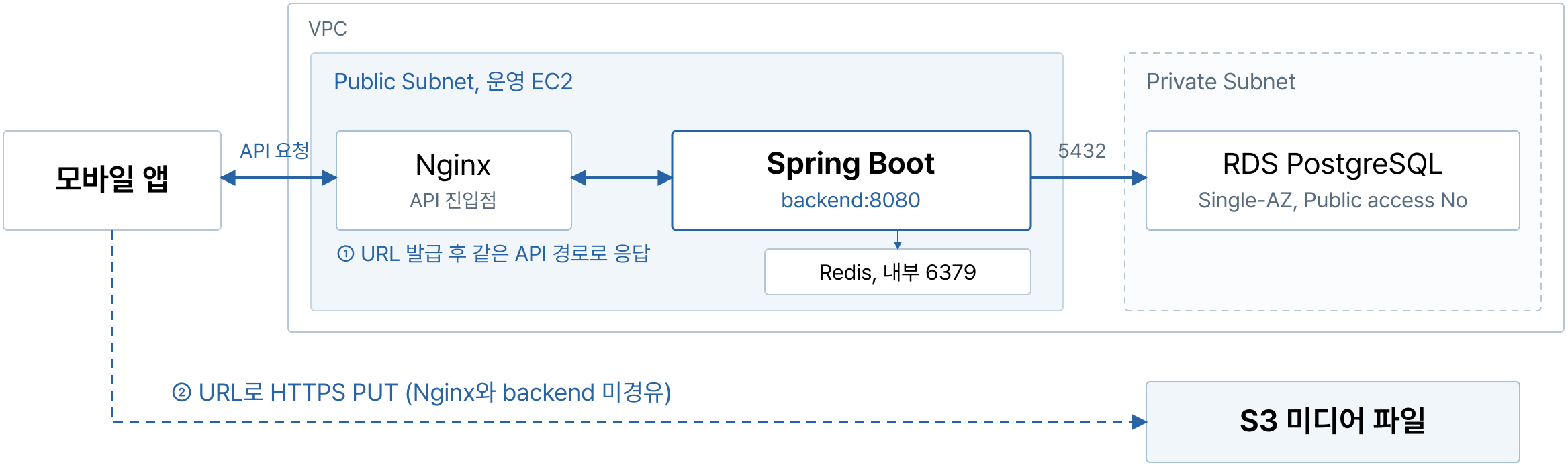
```
assertThat(signedHeaders)
  .contains("content-type")
  .doesNotContain("content-length");
```

S3 URL 발급 Provider 테스트 3건 통과 (2026.08.08)

API와 S3의 요청 경로 → p.07

API 요청과 S3 직접 PUT

팀 서비스의 API와 파일 전송 경로



단일 EC2와 Single-AZ RDS 구성입니다. 두 Private Subnet은 DB Subnet Group을 구성합니다.

AWS 구성과 복구 조건 → p.15

backend 배포와 복구 절차 → p.08

백엔드 교체와 실패 시 이미지 복구 절차 구현

배포 버전과 복구 대상을 식별하도록 40자리 SHA를 사용하고, Nginx와 Redis는 유지했습니다.

Jenkins, CI용 EC2
ECR에 Commit SHA 이미지 Push

→

SSM Run Command
운영 EC2에서 deploy.sh 호출

CI용 EC2와
팀 운영 AWS 환경 분리

01 **backend 교체**
ECR Pull
이전 이미지 보관

→

02 **backend health**
컨테이너 내부 검사

→

03 **외부 health**
Nginx reload 후
외부 경로 확인

오류 ↓

이전 이미지로 재기동 후 같은 순서로 검사
DB 스키마 복구는 별도 처리.

배포 절차를 팀 서비스에 적용했습니다. Health Check 실패 시 이전 이미지로 복구하는 절차를 구현했습니다.

이미지 복구 조건 → p.15

← 배포 대상 backend의 요청 경로 p.07

INFRASTRUCTURE

잘타슈

대전 공공자전거의 현재 현황과 도착 시점의 예측을 보여주는 서비스

서비스	대여소 조회, 예상 잔여 대수, 이동 경로
기간	2026.08.18 – 09.28
팀	6명 (AI+FE 2, BE 2, DATA 1, INFRA 1) 인프라 담당

직접 맡은 작업

서버 구성과 CI/CD, 데이터 처리 실행 환경

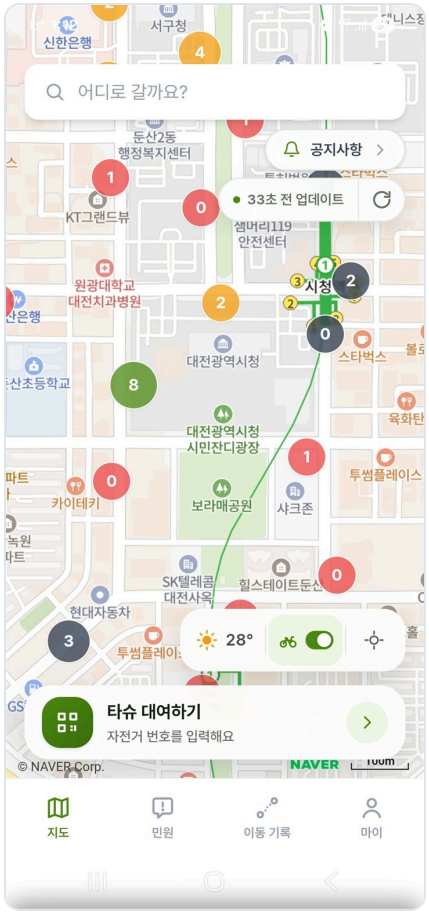
승인형 배포와 실패한 후보 정리

Jenkins 설정 복구, 백업과 격리 복원

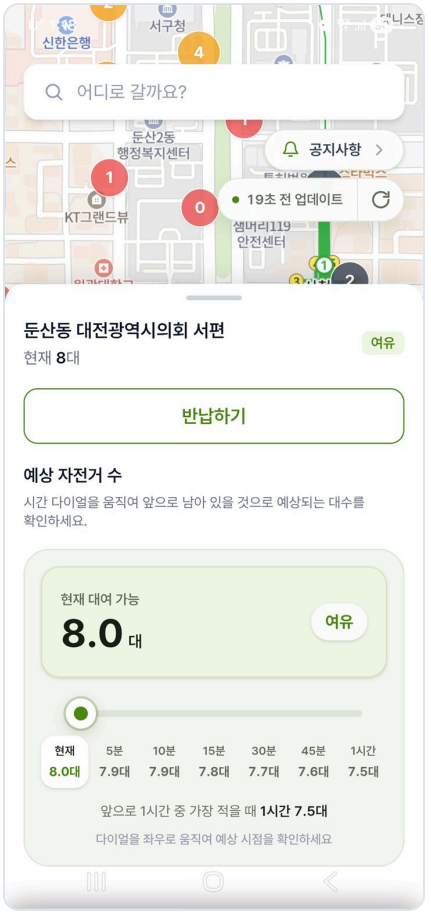
직접 구성한 인프라: EC2, ECR, S3, Docker, Jenkins, Nginx

MySQL, Redis, Prometheus, Grafana, Loki

팀 모듈의 실행 환경과 연결: Kafka, Spark, AI Serving



대여소의 현재 현황



도착 시점의 예상 잔여 대수

잘타슈 서버 구성과 데이터 흐름

팀원이 개발한 Kafka/Spark 처리 모듈과 서비스를 두 EC2에 배치했습니다. Docker 실행 환경, 사설 연결과 접근 권한을 구성하고, 합의한 Redis DB 구분을 반영했습니다.

Server A Nginx, Frontend / Backend, Collector
MySQL, Redis, OSRM

Server B Kafka, Spark, AI Serving
Jenkins → ECR → A 배포 / Prometheus, Grafana, Loki

현재 상태	Collector A	→ Redis DB 0 A	→ Backend A	→ 사용자 화면
이력 적재	Collector A	→ Kafka B	→ Spark B	→ MySQL A
예측 조회	MySQL A	→ AI Serving B	→ Redis DB 2 A	→ Backend A
경로 조회	Backend A	⇄ OSRM A	경로 요청과 응답	

→ 데이터 전달, ⇄ 경로 요청과 응답

단일 Kafka broker, 운영 Spark Worker 1개
두 서버는 역할 분리이며 서비스 이중화가 아닙니다.

Redis 연결과 승인 설정 → p.16

후보를 먼저 확인하고, 승인 후 트래픽 전환

후보 상태를 확인한 뒤 운영에 반영하도록, 최초 배포부터 새 버전 기동과 트래픽 전환을 분리했습니다.

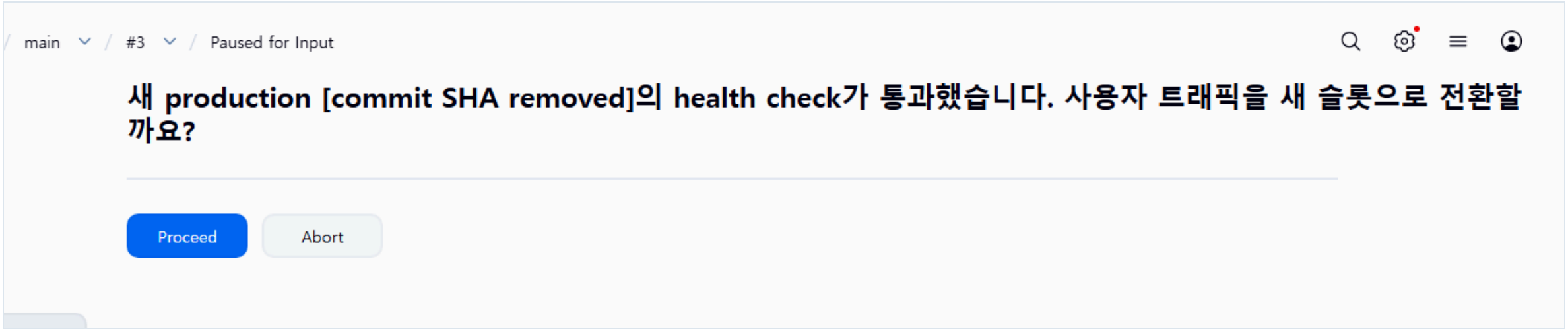
01
후보 기동
비활성 슬롯에
SHA 이미지 배포

→ 02
health 확인
후보 상태 검사
기존 트래픽 유지

→ 03
수동 승인
승인 stage 제한 15분
승인 뒤 상태 재확인

→ 04
Nginx 전환
운영 요청을
새 슬롯으로 연결

→ 05
외부 확인
health 실패 시
이전 대상으로 복구



Jenkins 수동 승인 대기 화면

운영 경계 Blue와 Green은 같은 Server A의 실행 슬롯입니다. 두 슬롯은 운영 DB를 공유합니다. 이미지 롤백으로 DB 변경까지 되돌릴 수 없어, 스키마 호환성은 별도로 확인해야 합니다.

승인 대기 설정 → p.16

실패 후보 정리 → p.12

승인 취소 후 남은 후보를 안전하게 정리

승인 취소 후 Green 후보가 남았습니다. 자동 정리를 추가하고 삭제 대상을 제한했습니다.

후보 상태가 없으면 종료

반복 실행을 no-op으로 처리

요청 SHA와 후보 SHA 비교

다른 배포가 만든 후보는 정리하지 않음

후보와 활성 슬롯 비교

현재 트래픽을 받는 슬롯은 삭제 거부

조건을 통과한 후보의
컨테이너와 상태 파일만 정리

활성 슬롯 삭제 거부

```
if [[ ${candidate_slot} == "${active_slot}" ]]; then
  echo 'candidate가 현재 활성 slot이므로 자동 정리를 거부합니다.'
>&2
  exit 1
fi
```

후처리 계정 수정

후처리의 SSH 계정이 달라 정리에 실패했습니다.
배포 계정과 일치하도록 수정했습니다.

수정 후 승인 취소 검증

후보 컨테이너와 상태 파일은 제거되고 활성 슬롯은 유지됐습니다.
Frontend ok, Backend UP을 확인했습니다.

승인 대기와 취소 후 정리 → p.16

사라진 자격증명, 선언형 설정으로 복구

01 **문제 발생**

Job과 빌드 기록은 유지됐지만,
자격증명이 제거돼 scan에 실패했습니다.

02 **원인 구분**

UI에서 등록한 자격증명이 JCasC 선언에 없어,
설정을 다시 적용할 때 제거됐습니다.

03 **설정 수정**

Secret 파일로 값을 공급하고 JCasC에 ID를
선언했습니다.
필수 값 누락 시 Jenkins 시작을 차단합니다.

04 **재시작 검증**

재시작 후 자격증명, Job, Agent가 유지됐고,
Polling 자동 빌드 성공을 확인했습니다.



JCasC 자격증명 선언

재시작 확인 결과 → p.17

```
- usernamePassword:
    scope: GLOBAL
    id: "gitlab-readonly"
    username: "${gitlab_username}"
    password: "${gitlab_token}"
```

MySQL 백업 생성과 격리 복원 검증

중복 실행과 미완성 파일 노출을 막고, 업로드 실패가 로컬 백업에 영향을 주지 않도록 구성했습니다.

로컬 백업

lock → 임시 파일 → gzip 검사
완료 후 rename과 checksum

S3 보관

별도 작업으로 업로드
전송 실패와 로컬 백업 분리

격리 복원

network none, tmpfs
SQL import 후 두 schema 확인

예방 장치와 복원 시험 결과

없는 MySQL 지정 → **완성 파일 없이 종료**

중복 실행은 lock으로 차단, **exit 75**

S3에서 재다운로드 후 **checksum 일치 확인**

업로드 인증 실패를 **로컬 백업과 분리**

격리 MySQL 실행

```
docker run -d \
  --name "${container_name}" \
  --network none \
  --tmpfs /var/lib/mysql:rw,noexec,nosuid,size=512m \
```

백업을 다시 내려받아 checksum을 비교하고, 격리 MySQL에 SQL을 import한 뒤 두 스키마의 존재를 확인했습니다.

백업과 격리 복원 결과 → p.17

팀 서비스에 적용한 AWS와 오류 시 복구 절차

프로젝트 기간에 구성한 AWS와 Health Check 실패 시 이전 이미지로 복구하도록 구현한 코드입니다.

AWS 구성

EC2	Public Subnet, IAM Role
RDS	Single-AZ, Public access No DB Subnet Group: 두 Private Subnet

AWS 리소스 접근

EC2 IAM Role로 S3, ECR, SSM 접근 권한을 구성했습니다. RDS의 5432 접근 출처는 EC2 보안 그룹으로 제한했습니다.

이전 이미지로 복구하는 조건

```
if [[ -n "$previous_image" ]] && APP_ROOT="$app_root"
HEALTH_CHECK_URL="$public_health_url" \
  bash "$app_root/infrastructure/production/scripts/rollback.sh"
"$previous_image"; then
  rm -f "$state_dir/pending-image"
  printf 'DEPLOY_STATUS=rollback-success\n'
  exit 42
fi
```

Redis 연결과 승인 대기 설정

Redis DB 연결 확인

담당자들과 Redis DB 구분을 합의하기 전에는 자동 배포를 막았습니다. 합의 후 AI 실행 이미지에서 DB 2 PING을 확인했습니다.

15분 제한은 승인 stage에 적용

```
stage('Approve production traffic switch') {
    when {
        branch 'main'
    }
    options {
        timeout(time: 15, unit: 'MINUTES')
    }
```

이 stage는 `input`으로 승인을 기다립니다.
전체 pipeline 제한 45분은 별도 상위 설정입니다.

승인 취소 후 후보 정리 동작을 확인했습니다 (p.12).

Jenkins 재시작과 백업 복원 결과

Jenkins 재시작 후 확인

Docker Secret으로 값을 공급하고 JCasC에 자격증명을 선언했습니다. 격리 검증 후 실제 Controller를 재시작해 설정이 유지되는지 확인했습니다.

확인 대상	결과
GitLab 자격증명	재시작 후 유지
Job과 기존 빌드 기록	재시작 후 유지
Build Agent	연결 유지
Polling 자동 빌드	중지 중 누적된 커밋을 감지하고 SUCCESS

백업 전송과 격리 복원 확인

확인 대상	결과
S3 재다운로드	백업과 checksum의 SHA-256 일치
격리 MySQL 복원	SQL import 후 운영 및 스테이징 스키마 존재 확인
업로드 자동 실행	timer 활성화, service 종료 코드 0
인증 실패 주입	업로드만 종료 코드 1로 실패. 운영 MySQL과 로컬 및 원격 백업 timer는 정상 유지